REVISED 28 OCT 2026

VisionIQ — User Stories by Phase

28 October 2026

How to read this

VisionIQ is built as 51 user stories (46 core and 5 optional extensions) covering all 236 backlog tasks, organised as **phase → feature → story → task**. Development readiness (G0) comes first, then the four v1.11 phases, then the work that supports them.

Order	Section	Stories	What it delivers
0	G0 — Development readiness	6	Everything needed before coding the phases
1	Phase 1 — Learn	14	Knowledge base from history, kept current from PR outcomes
2	Phase 2 — Brief	8	Evidence-backed brief on every ticket
3a	Phase 3a — Defect	6	Reproduced bug, verified fix, draft PR
3b	Phase 3b — Enhancement	2	Build guidance, optional starter PR
4	Phase 4 — Human Gate	2	Engineer decides; advice tracked right or wrong
—	Impact Agent	2	History view inside the flagship tool
—	Quality and Demo	4	Measured accuracy, safety tests, live demo
—	Reporting	2	What was done and how accurately (delivery: D11)
—	Extensions (optional)	5	Jira, OKF, ADK, wider coverage, workspace — only after the win condition works

Learn is not one-time: it builds history first, then keeps learning from PRs and their outcomes.

Story card fields

Field	Meaning
Feature	The capability the story belongs to, with its v1.11 capability group (A–H)
Gate · Priority	v1.11 delivery gate (G0–G5). *Vertical slice* = build first, the minimum end-to-end path on one repo. *Demo-critical* = also needed for the win condition. *After demo*, *Pending D11*, *Extension* = later, waiting on a decision, or optional
Stack	The kind of work in this story; each task lists its own technology
Environment	Access, tools or decisions needed from outside the code; readiness checks ENV-01 to ENV-18 are in G0
Depends on	Stories that must be finished first
v1.11	Section of the master design this story implements
Acceptance criteria	What must be true to call the story done
Tasks	The technical work, with its technology and backlog ID (VisionIQ_Master_Backlog.xlsx); Proposed = not in the backlog yet, needs approval (listed in Proposed tasks)
Completion evidence	What proves it is done: a test, a run, a PR link or a demo step

Status is tracked in the backlog, not here. Source of truth: VisionIQ Master Engineering Design v1.11; if a story and v1.11 disagree, v1.11 wins.

Capability groups A–H come from v1.11 §4. That table lists feature F09 twice and omits F14, so stories are not mapped to individual F-numbers until the original feature list is confirmed.

G0 — Development readiness

G0 is done when the six foundation stories pass. Not every readiness check is needed before the first line of code: each one says when it is needed and how to prove it is done.

Readiness checks

ID	When needed	What to prepare	Technology	Proof it is done	Story
ENV-01	Before any coding	.NET SDK and IDE installed	.NET 8, ASP.NET Core	dotnet build and /health succeed	FND-02
ENV-02	Before any coding	Python and its dependency environment	Python 3.11, FastAPI, uv or venv	pytest smoke test and /health succeed	FND-02
ENV-03	Before any coding	Repository cloned, with branch push rights	Git, Azure DevOps Repos	Clean checkout and a pushed branch	FND-02
ENV-04	Before Learn work	Local PostgreSQL with pgvector and migrations	Docker, PostgreSQL, pgvector, EF Core, Alembic	Migrations run on a fresh database; vector smoke test passes	FND-03
ENV-05	Before connecting sources	Authorized access to the demo repo, tickets, PR and CI history, Confluence spaces	ADO REST, Git, Confluence API	One full ticket → PR → commit → test chain read	FND-01, LRN-01
ENV-06	Before service integration	Event and API contracts agreed	JSON Schema, OpenAPI	Contract fixtures pass from the .NET producer to the Python consumer	FND-05
ENV-07	Before any model call	Data policy for Gemini and Claude (D04)	Vertex AI, Anthropic	Written list of allowed data, retention and egress	BRF-02
ENV-08	Before building the vector index	Embedding model and dimension pinned (D08)	Vertex AI embeddings, pgvector	Model ID and dimension recorded; test embedding stored	LRN-08
ENV-09	Before cloud deployment	Google Cloud project, billing and budgets	Google Cloud	APIs enabled; budget alert tested	FND-04
ENV-10	Before cloud deployment	Identities, secrets and network policy	IAM, Secret Manager, VPC	Least-privilege test passes; runner secret read is denied	FND-04
ENV-11	Before cloud integration	Cloud SQL, Pub/Sub with dead-letter, Cloud Storage	Cloud SQL, Pub/Sub, Cloud Storage	Migration and publish–consume–retry smoke test pass	FND-03, FND-05
ENV-12	Before defect work	Demo repo and approved test runner (D03)	Git, runner image	The defect's failing test reproduces with the allowed command	FND-01, DEF-01
ENV-13	Before the PR demo	Bot identity, branch policy and PR pipeline	ADO permissions, Azure Pipelines	Bot merge attempt rejected; PR pipeline reports on the latest commit	HG-01, DEF-06
ENV-14	Investigate in week 1; needed for the final demo	Impact Agent integration point (D02)	Impact Agent	Authenticated test call, or a recorded blocker	IA-01
ENV-15	Before final evaluation	Labelled past tickets and a touched-path workload (D09, D10)	pytest, benchmark	Cases labelled with ground truth; benchmark method written	QA-01, DEF-03

ID	When needed	What to prepare	Technology	Proof it is done	Story
ENV-16	Before the enhancement demo	Live enhancement ticket with history	ADO, Git	Ticket and an affected contract example available	ENH-01
ENV-17	Before report publication, not before coding	Reporting delivery (D11)	Decided by D11	Decision recorded; delivery tested with judge access	RPT-02
ENV-18	Optional, after the core path	Decide on ADK and OKF by measured value	Google ADK, OKF	Comparison recorded; the core works with both disabled	EXT-02, EXT-03

Still open, not blocking G0 code: embedding model (D08), Impact Agent interface (D02), reporting delivery (D11).

Feature: Scope and decisions

FND-01 · Agree scope, live tickets and decisions

Field	Detail
Feature	Scope and decisions
Gate · Priority	G0 · Vertical slice
Stack	Planning
Environment	Sandbox access from the organizers
Depends on	None
v1.11	§1.3, §17, §19

Technical story: agree demo scope, choose live tickets and close blocking decisions, so the team builds the same thing.

Acceptance criteria

- One live open defect and one live open enhancement are chosen, in an area with past fixes and a runnable test.
- A past fix with a later revert is identified for the learning demonstration.
- Each clause of the win statement has a matching acceptance check.
- Decisions D01–D07 are recorded as ADRs.

Tasks

Task	Technology	ID
Capture organizer requirement traceability	Document	S00-01
Choose sandbox projects, repos and live tickets	ADO, Git	S00-02
Record demo constraints and exclusions	Document	S00-04
Define measurable win demonstration	Document	S00-05
Establish demo evidence register	Document	S00-08
Create architecture decision register	ADR files in repo	S01-10

Completion evidence: ticket IDs and the ADR register committed to the repository.

Feature: Development environment

FND-02 · Repositories and local development

Field	Detail
Feature	Development environment
Gate · Priority	G0 · Vertical slice
Stack	.NET, Python, DevOps
Environment	.NET 8 SDK, Python 3.11, Docker on each machine
Depends on	FND-01
v1.11	§8.2

Technical story: set up the .NET and Python services so each builds, tests and runs locally from a fresh checkout.

Acceptance criteria

- A clean checkout builds and starts both services with one command.
- Lint and unit tests run on every PR and block the merge when they fail.

Tasks

Task	Technology	ID
Create version-controlled solution repository	Git, Azure DevOps Repos	S01-01
Bootstrap ASP.NET Core service	.NET 8, ASP.NET Core	S01-02
Bootstrap Python intelligence service	Python, FastAPI	S01-03
Local service orchestration	Docker Compose	S01-04
Code style and lint policy	.NET analyzers, Ruff	S01-06
Unit-test harnesses	xUnit, pytest	S01-07
PR validation workflow	Azure Pipelines	S01-09

Completion evidence: a green pipeline run on a fresh PR.

FND-03 · Database with clear ownership

Field	Detail
Feature	Development environment
Gate · Priority	G0 · Vertical slice
Stack	Database
Environment	Decision D07 (schema ownership)
Depends on	FND-02
v1.11	§6.13, §10.8

Technical story: one PostgreSQL database with three schemas (workflow, evidence, knowledge), each written only by the service that owns it.

Acceptance criteria

- pgvector and full-text search are enabled locally and in Cloud SQL.
- Migrations run cleanly on an empty database.
- Each service can read the other schemas but a write to another service's schema is rejected.

Tasks

Task	Technology	ID
Local database migrations for three schemas	PostgreSQL, EF Core, Alembic	S01-05
Provision Cloud SQL	Cloud SQL, Terraform	S02-05
Enforce schema write isolation	PostgreSQL roles	S02-15

Completion evidence: migration log on a fresh database and a failing cross-schema write test.

Feature: Cloud platform

FND-04 · Google Cloud environment

Field	Detail
Feature	Cloud platform
Gate · Priority	G0 · Vertical slice
Stack	GCP / DevOps
Environment	Google Cloud project with billing; permission to create service accounts
Depends on	FND-02
v1.11	§9, §10.8

Technical story: a Google Cloud project where both services deploy, with one identity per role and secrets kept out of code.

Acceptance criteria

- Both services deploy to Cloud Run from the pipeline, not by hand.
- Source read, model call, code run, PR creation and deploy each use their own identity.
- The test runner identity cannot read any secret.

Tasks

Task	Technology	ID
Local secrets template	env file, .gitignore	S01-08
GCP project and budgets	Google Cloud, Billing	S02-01
Terraform modules	Terraform	S02-02
Service identities	IAM	S02-03
Artifact Registry	Artifact Registry	S02-04
Cloud Storage buckets	Cloud Storage	S02-06
Secret Manager	Secret Manager	S02-09
Deploy two Cloud Run services	Cloud Run, Cloud Build	S02-10
Private network egress	VPC, Cloud NAT	S02-12
IaC promotion flow (after demo)	Terraform	S02-13

Completion evidence: both service URLs return healthy, and a denied secret read by the runner identity.

FND-05 · Reliable events between .NET and Python

Field	Detail
Feature	Cloud platform
Gate · Priority	G0–G1 · Vertical slice
Stack	Contracts, .NET, Python

Field	Detail
Environment	None beyond FND-04
Depends on	FND-02, FND-04
v1.11	§8.1, §8.2

Technical story: the two services talk through versioned events on Pub/Sub, so nothing is lost, duplicated or misread.

Acceptance criteria

- Every event follows a versioned schema and carries a correlation ID.
- A duplicate event changes nothing; an event that keeps failing lands in the dead-letter queue.
- One request can be traced across both services' logs.

Tasks

Task	Technology	ID
Shared event envelope	JSON Schema	S03-01
analysis.requested contract	JSON Schema	S03-02
analysis.completed contract	JSON Schema	S03-03
PR event and learning contracts	JSON Schema	S03-04
Python internal APIs	FastAPI, OpenAPI	S03-05
Typed .NET client and Python models	C# records, Pydantic	S03-06
Inbox idempotency	.NET, PostgreSQL	S03-07
Outbox and event publishing	.NET, Pub/Sub	S03-08
Retry and dead-letter strategy	Pub/Sub	S03-09
Correlation and provenance	Cloud Logging	S03-10
Pub/Sub topics	Pub/Sub, Terraform	S02-07
Dead-letter and retry configuration	Pub/Sub	S02-08
Contract version migration test (after demo)	JSON Schema, CI	S03-11

Completion evidence: a test publishing the same event twice with one stored result, and one trace ID visible in both services' logs.

FND-06 · Jobs, monitoring and limits

Field	Detail
Feature	Cloud platform
Gate · Priority	G0 for jobs, G5 for hardening · Demo-critical
Stack	GCP / DevOps
Environment	None beyond FND-04
Depends on	FND-04
v1.11	§9, §10.8

Technical story: background jobs, monitoring, data retention and rate limits, so the system runs reliably through the demo.

Acceptance criteria

- The backfill runs as a scheduled Cloud Run job.
- Logs and metrics from both services are in one place.
- Azure DevOps and model rate limits are respected without failing runs.

Tasks

Task	Technology	ID
Deploy backfill job	Cloud Run Jobs, Cloud Scheduler	S02-11
Centralized observability	Cloud Logging, Cloud Monitoring	S02-14
Structured telemetry	OpenTelemetry	S16-01
Data retention and privacy	Cloud Storage lifecycle, PostgreSQL	S16-04
Rate-limit and quota protection	.NET, Python	S16-06
Ingestion and job monitoring (after demo)	Cloud Monitoring	S16-02
LLM cost accounting (after demo)	Cloud Billing	S16-03
Backup and restore rehearsal (after demo)	Cloud SQL	S16-05
Dead-letter replay runbook (after demo)	Pub/Sub	S16-07
Incident and on-call runbook (after demo)	Document	S16-08
Verify cost and capacity (after demo)	Cloud Billing	S16-09

Completion evidence: a scheduled backfill run in the job history and one dashboard showing both services.

Phase 1 — Learn

Learn turns past tickets, PRs, commits, builds and design pages into a permission-aware knowledge base, then keeps it current from every PR and its outcome. Every other phase reads from it. 14 stories in 6 features, gates G1–G3 (v1.11 §6, §8.12).

Feature: Source connectivity

LRN-01 · Connect to the history sources

Field	Detail
Feature	Source connectivity (capability A)
Gate · Priority	G1 · Vertical slice
Stack	.NET
Environment	Read access to the sandbox ADO projects and repositories
Depends on	FND-04, FND-05
v1.11	§6.6 stages 1–3

As an engineer picking up a ticket, *I want* VisionIQ to read every past ticket, PR, commit and build in my project, *so that* its advice is based on what actually happened.

Acceptance criteria

- Work items, PRs, commits, changed files and CI runs are read for every approved project.
- A source that can't be reached is marked NOT SYNCED, never shown as empty.
- An interrupted sync resumes from its last checkpoint without duplicates.

Tasks

Task	Technology	ID
Inventory historical data and connectors	ADO, Git	S00-03
Read ADO projects and work items	.NET, ADO REST (WIQL)	S04-01
Read ADO PR metadata	.NET, ADO REST	S04-04
Read Git repository metadata	.NET, LibGit2Sharp	S04-10
Extract PR and commit links	.NET	S04-11
Read CI runs and artifacts	.NET, ADO Pipelines REST	S04-12
Connector checkpoints	.NET, PostgreSQL	S04-14
Connector capability registry	.NET	S04-15

Completion evidence: sync report with counts per source, and a resumed run with no duplicate rows.

LRN-02 · Include design decisions from Confluence

Field	Detail
Feature	Source connectivity (capability A)
Gate · Priority	G1 · Demo-critical
Stack	.NET
Environment	Read access to the approved Confluence spaces
Depends on	LRN-01
v1.11	§6.20

As an engineer, *I want* relevant Confluence design pages in the history, *so that* advice reflects decisions already made, not just code.

Acceptance criteria

- Pages are stored with version and page permissions.
- A page the engineer can't open in Confluence is never shown to them through VisionIQ.

Tasks

Task	Technology	ID
Confluence page snapshot connector	.NET, Confluence REST	S04-08
Confluence permission mapping	.NET	S04-09
Normalize Confluence and design decisions	.NET	S05-07

Completion evidence: an imported page with its version, and a restricted page hidden from an unauthorized user.

Feature: Canonical knowledge base

LRN-03 · Store history as versioned evidence

Field	Detail
Feature	Canonical knowledge base (capability A)
Gate · Priority	G1 · Vertical slice
Stack	.NET, Database
Environment	None beyond G0
Depends on	FND-03, LRN-01
v1.11	§6.1, §6.6 stage 4

Technical story: store every source item as a versioned, attributed record with its permissions, so history can be trusted and audited.

Acceptance criteria

- Each record keeps its source ID, revision, timestamps and access list.
- A raw copy is kept in Cloud Storage; sensitive fields are redacted before indexing.
- Re-importing the same item updates it instead of creating a duplicate.

Tasks

Task	Technology	ID
Define canonical evidence schema	PostgreSQL, EF Core	S05-01
Define source identity strategy	PostgreSQL	S05-02
Raw immutable snapshot archive	.NET, Cloud Storage	S05-03
Normalize work-item revisions	.NET	S05-04
Normalize PR, commit and review artifacts	.NET	S05-05
Normalize CI, deployment and incident artifacts	.NET	S05-06
Deduplication and upsert	.NET, PostgreSQL	S05-11

Completion evidence: the same item imported twice leaves one record with two revisions.

LRN-04 · Link each ticket to the fix that closed it

Field	Detail
Feature	Canonical knowledge base (capability A)

Field	Detail
Gate · Priority	G1 · Vertical slice
Stack	.NET
Environment	None beyond G0
Depends on	LRN-03
v1.11	§6.6 stage 6, §6.14

As an engineer, *I want* to see which PRs and commits fixed a past defect, and whether that fix was later reverted, *so that* I learn from what really worked.

Acceptance criteria

- Links come only from explicit references; time proximity alone never creates a link.
- Each link is marked explicit, inferred, verified, disputed or unknown.
- One ticket can link to several fixes and to a later revert.

Tasks

Task	Technology	ID
Deterministic explicit links	.NET	S05-08
Inferred association store	.NET, PostgreSQL	S05-09
Unresolved and disputed evidence handling	.NET	S05-10

Completion evidence: five known ticket → fix links reviewed by the team, all correct or the errors logged.

LRN-05 · Load full history and keep it current

Field	Detail
Feature	Canonical knowledge base (capability A)
Gate · Priority	G1 · Vertical slice
Stack	.NET, Python
Environment	None beyond G0
Depends on	LRN-03, LRN-04, FND-06
v1.11	§6.6 stages 2–3

As an engineer, *I want* the knowledge base to include years of history and today's changes, *so that* the brief never misses recent work.

Acceptance criteria

- The initial backfill completes for all approved projects, with counts per source.
- New and changed items arrive through events plus a periodic reconciliation.
- Failed items go to a quarantine list instead of stopping the run.

Tasks

Task	Technology	ID
Build initial source backfill	.NET, Cloud Run Jobs	S05-12
Delta and reindex mechanism	Python	S05-13
Ingestion audit and dead-letter queue	.NET, Pub/Sub	S05-14
Verify historical data completeness	Testing	S05-15

Completion evidence: backfill report, and a changed ticket appearing within one sync cycle.

Feature: Code intelligence and graph

LRN-06 · Understand the code

Field	Detail
Feature	Code intelligence (capability B)
Gate · Priority	G1 · Vertical slice
Stack	Python
Environment	Decision D03 (demo repo and its language)
Depends on	LRN-03
v1.11	§4.7, §6.17

As an engineer, *I want* VisionIQ to know which files, functions and dependencies a change touches, *so that* it can warn me about related areas.

Acceptance criteria

- The demo repo is scanned into files, symbols and dependencies.
- A language without a parser falls back to file-level history; symbols are never invented.
- A parser crash marks that repo PARTIAL and keeps the last good result.

Tasks

Task	Technology	ID
Repository scanner	Python, Git	S06-03
Parser-adapter interface	Python	S06-04
File-level fallback	Python	S06-05
Parser for the demo repo language	Python, tree-sitter	S06-06
Resolve dependency relationships	Python	S06-07
Map diff hunks to symbols	Python	S06-08
Cross-project dependency associations	Python	S06-10
Verify parser failure isolation	pytest	S06-13

Completion evidence: symbol list for the demo repo, and a forced parser crash leaving the last good result.

LRN-07 · Build the engineering relationship graph

Field	Detail
Feature	Code intelligence (capability B)
Gate · Priority	G1 · Demo-critical
Stack	Database, .NET, Python
Environment	None beyond G0
Depends on	LRN-04, LRN-06
v1.11	§6.18

Technical story: connect tickets, PRs, commits, files, tests and incidents in one graph, so VisionIQ can follow a path from a symptom to past causes.

Acceptance criteria

- Graph nodes and edges follow one agreed taxonomy.
- Renames and reverts update edges without losing history.
- Graph queries return only items the requester may see.

Tasks

Task	Technology	ID
Define graph node and edge taxonomy	PostgreSQL	S06-01
Create graph nodes and edges schema	PostgreSQL	S06-02
Graph edge lifecycle	.NET	S06-09
Graph ACL filtering	Python, SQL	S06-11
Graph query APIs	Python, FastAPI	S06-12

Completion evidence: a query from a defect to its fix and test, and the same query hiding a restricted node.

Feature: Search

LRN-08 · Index history for search

Field	Detail
Feature	Search (capability B)
Gate · Priority	G2 · Vertical slice
Stack	Python
Environment	Vertex AI access; decision D08 (embedding model)
Depends on	LRN-05
v1.11	§6.6 stage 7, §10.8

Technical story: index history for keyword and meaning-based search, so a ticket matches past work even when the words differ.

Acceptance criteria

- Every chunk keeps its source, revision and access list.
- The embedding model and dimension are pinned and stored with each vector.
- Keyword and vector search both return results on the demo data.

Tasks

Task	Technology	ID
Define retrieval chunk strategy	Python	S07-01
Set up embedding model and dimensions	Vertex AI embeddings	S07-02
Embedding batch jobs	Python, Cloud Run Jobs	S07-03
Create lexical index	PostgreSQL full-text	S07-04
Create pgvector similarity index	pgvector	S07-05

Completion evidence: one query answered by both keyword and vector search over the demo history.

LRN-09 · Find the right history for a ticket

Field	Detail
Feature	Search (capability B)
Gate · Priority	G2 · Vertical slice
Stack	Python
Environment	None beyond G0

Field	Detail
Depends on	LRN-07, LRN-08
v1.11	§7.1

As an engineer, *I want* VisionIQ to find the past tickets and fixes most like mine, *so that* I start from evidence instead of from scratch.

Acceptance criteria

- Keyword, vector and graph results are combined and ranked.
- Access is checked before and after retrieval; revoked evidence never appears.
- When evidence is weak, VisionIQ says so instead of guessing.

Tasks

Task	Technology	ID
Graph candidate retrieval	Python, SQL	S07-06
Parallel candidate union	Python	S07-07
Scope and ACL prefilter and post-check	Python	S07-08
Dedupe and reranking	Python	S07-09
Grounded snippet rendering	Python	S07-10
Ambiguity and abstention policy	Python	S07-11
Benchmark retrieval speed and quality	pytest	S07-13
Retrieval cache and invalidation (after demo)	Python	S07-12

Completion evidence: ranked results for the live defect, and an abstention on a deliberately vague query.

Feature: Knowledge qualification

LRN-10 · Turn repeated failures into lessons

Field	Detail
Feature	Knowledge qualification (capability C)
Gate · Priority	G1 · Demo-critical
Stack	Python
Environment	None beyond G0
Depends on	LRN-04
v1.11	§6.10, §6.16 KB-4

As an engineer, *I want* repeated failures grouped into one lesson with its evidence, *so that* I see the pattern, not ten similar tickets.

Acceptance criteria

- A lesson is created only when evidence qualifies it; a missing cause stays unverified.
- A new observation joins the existing pattern instead of creating a duplicate.
- A later revert disputes the lesson and keeps its history.

Tasks

Task	Technology	ID
Define canonical lesson schema	PostgreSQL, Alembic	S09-01
Define lesson creation qualification	Python	S09-02
Pattern candidate clustering	Python	S09-03

Task	Technology	ID
Pattern merge and dispute flow	Python	S09-04
Verify non-duplicative learning	pytest	S09-11

Completion evidence: three similar tickets linked to one lesson, and a revert marking that lesson disputed.

LRN-11 · Prove the knowledge base can be trusted

Field	Detail
Feature	Knowledge qualification (capability C)
Gate · Priority	G1 · Demo-critical
Stack	Testing
Environment	None beyond G0
Depends on	LRN-05, LRN-10
v1.11	§6.16 KB-5, §11.1.4

Technical story: run the knowledge-base readiness tests from v1.11, so Brief work starts on history the team trusts.

Acceptance criteria

- A revoked item cannot be read through canonical evidence queries.
- A ticket with two PRs, one reverted, shows both without claiming a verified cause.
- A connector outage serves the last good snapshot or fails closed.

Tasks

Task	Technology	ID
Test source permission revocation	pytest	S15-15
Run knowledge base readiness tests	pytest, fixtures	S15-16

Completion evidence: saved test results with fixture, snapshot ID and assertion for each case.

Feature: Continuous learning

LRN-12 · Every PR event updates the knowledge base

Field	Detail
Feature	Continuous learning (capabilities D, G)
Gate · Priority	G2 · Demo-critical
Stack	.NET
Environment	ADO service hooks for PR events
Depends on	LRN-01, FND-05
v1.11	§8.12

As an engineer, *I want* new PRs, reviews and merges added to history as they happen, *so that* the next brief already knows about them.

Acceptance criteria

- PR created, updated, reviewed, abandoned and merged events are each stored once.
- Events arriving late or out of order end in the same result.
- No PR event marks a root cause as verified on its own.

Tasks

Task	Technology	ID
Define PR lifecycle state model	.NET	S12-01
Capture PR created event	.NET, ADO service hooks	S12-02
Capture PR updated and review event	.NET, ADO service hooks	S12-03
Capture PR abandoned or rejected	.NET	S12-04
Capture PR merged	.NET	S12-05
Event ordering and replay	.NET, PostgreSQL	S12-14

Completion evidence: the same PR event replayed twice with one stored result.

LRN-13 · Lessons change only on proof

Field	Detail
Feature	Continuous learning (capabilities D, G)
Gate · Priority	G2–G3 · Demo-critical
Stack	Python
Environment	A past fix and later revert chosen in FND-01
Depends on	LRN-12, LRN-10
v1.11	§8.9, §8.12

As an engineer, *I want* lessons updated only when new evidence confirms or disputes them, *so that* the knowledge base doesn't drift with every PR.

Acceptance criteria

- A PR event re-indexes only the files and chunks it touched.
- A lesson gets a new version only when evidence materially changes; history is kept.
- The demo shows learning by replaying a real fix-then-revert from history.

Tasks

Task	Technology	ID
Targeted reindex on PR	Python	S12-11
Selective pattern consolidation	Python	S12-12
Publish knowledge.changed event	Python, Pub/Sub	S12-15
PR-learning demonstration fixtures	pytest, fixtures	S12-16

Completion evidence: replayed fix-then-revert showing the lesson move from supported to disputed, with both versions kept.

LRN-14 · Learn from outcomes after merge (after demo)

Field	Detail
Feature	Continuous learning (capabilities D, G)
Gate · Priority	After · After demo
Stack	.NET, Python
Environment	Access to deployment and incident records
Depends on	LRN-12, LRN-13
v1.11	§3.6, §8.8

As a team lead, *I want* deployments, reverts, reopened tickets and incidents linked back to the PR, *so that* lessons reflect what happened in production.

Acceptance criteria

- Each merged PR is watched for an agreed observation window.
- Outcomes are classified: verified effective, regressed or reverted, disputed, unknown, observing.
- Human review is required before a lesson becomes an enforceable rule.

Tasks

Task	Technology	ID
Deployment and incident event adapter	.NET	S04-13
Capture deployment evidence	.NET	S12-06
Capture revert or ticket reopen	.NET	S12-07
Capture production incidents	.NET	S12-08
Post-merge observation window	.NET	S12-13
Human lesson review workflow	Python	S09-10

Completion evidence: one merged PR with its recorded outcome class.

Phase 2 — Brief

Brief puts an evidence-backed summary on every defect and enhancement ticket as soon as it is picked up, without waiting for code or CI. 8 stories in 4 features, gate G2 (v1.11 §3.5, §7.4, §8.1, §10.9).

Feature: Ticket pickup

BRF-01 · Pick up tickets automatically

Field	Detail
Feature	Ticket pickup (capability D)
Gate · Priority	G2 · Vertical slice
Stack	.NET
Environment	ADO service hook permission; decision D05 (trigger state)
Depends on	FND-05, LRN-01
v1.11	§3.5, §8.1

As an engineer, *I want* VisionIQ to start as soon as I move a ticket to In Progress, *so that* I never have to open a separate tool.

Acceptance criteria

- The webhook is verified, stored and acknowledged within seconds; analysis runs in a worker.
- The same ticket event never starts a second run.
- A missed webhook is picked up by the reconciliation sweep.

Tasks

Task	Technology	ID
ADO service-hook receiver	ASP.NET Core, ADO service hooks	S04-02
Eligible ticket trigger rules	.NET	S10-01
Test missed-event recovery	xUnit, integration test	S15-14

Completion evidence: a ticket moved to In Progress starts exactly one run; a dropped webhook is recovered by the sweep.

Feature: AI model use

BRF-02 · Use Gemini and Claude safely

Field	Detail
Feature	AI model use (capability C)
Gate · Priority	G2 · Vertical slice
Stack	Python · AI models
Environment	Vertex AI access; decision D04 (what data may go to Claude)
Depends on	FND-04
v1.11	§10.9

Technical story: route each AI task to Gemini or Claude through one interface, with budgets, versioned prompts and injection protection, so model output is controlled and traceable.

Acceptance criteria

- Gemini drafts briefs; Claude handles code and conflicting evidence, per the v1.11 routing table.

- Every call logs provider, model, prompt version and tokens.
- Instructions hidden in ticket text are treated as data and flagged.
- When the budget runs out, the result is PARTIAL with the evidence found, never invented advice.

Tasks

Task	Technology	ID
Provider-neutral LLM interface	Python	S08-01
Gemini provider	Vertex AI SDK	S08-02
Claude provider	Anthropic SDK or Claude on Vertex (per D04)	S08-03
Task-routing decision table	Python	S08-04
Bounded budget and fallback	Python	S08-05
Prompt templates and versioning	Python	S08-06
Tool allowlist and prompt-injection isolation	Python	S08-07
Knowledge and evidence tool wrappers	Python	S08-08
Review and escalation rule	Python	S08-10
Cross-provider evidence consistency	Python	S08-11
Evaluate LLM output quality	pytest, labelled cases	S08-12
Test prompt injection and data leakage	pytest	S15-10

Completion evidence: call log showing provider and prompt version per request, and an injection test ticket flagged, not obeyed.

Feature: Brief content

BRF-03 · Brief with risk areas, do / don't and best approach

Field	Detail
Feature	Brief content (capabilities C, G)
Gate · Priority	G2 · Vertical slice
Stack	Python · AI models
Environment	None beyond G0
Depends on	LRN-09, BRF-02
v1.11	§7.4, §7.5

As an engineer picking up a defect, *I want* a brief showing past fixes, risky areas and what to avoid, *so that* I don't repeat mistakes others already made.

Acceptance criteria

- The brief lists risk areas, do / don't from past fixes and backfires, and a recommended approach.
- Risk is shown as LOW, MODERATE, HIGH or INSUFFICIENT_DATA, with the factors behind it.
- Missing data is never shown as low risk.

Tasks

Task	Technology	ID
Scope prediction	Python	S10-02
Deterministic risk factors	Python, SQL	S10-03
Structured brief schema	Python, JSON Schema	S10-04
Generate defect brief	Python, Gemini	S10-05

Task	Technology	ID
Explainable risk band	Python	S10-13

Completion evidence: the brief for the live defect, with its risk band and the factors behind it.

BRF-04 · Every point has a source

Field	Detail
Feature	Brief content (capability C)
Gate · Priority	G2 · Vertical slice
Stack	Python · AI models
Environment	None beyond G0
Depends on	BRF-03
v1.11	§7.4.1

As an engineer, *I want* every point in the brief to link to the ticket, PR or commit it came from, *so that* I can check it myself.

Acceptance criteria

- Each advice item cites at least one source; unsupported text is removed or marked.
- Each advice item has its own ID, so it can be rated later.
- When history is thin, the brief says so instead of guessing.

Tasks

Task	Technology	ID
Validate per-advice evidence	Python	S10-07
Handle missing or inconclusive evidence	Python	S10-10

Completion evidence: every advice item in the live defect brief resolves to a real source link.

Feature: Delivery in Azure DevOps

BRF-05 · Brief appears on the ticket

Field	Detail
Feature	Delivery in Azure DevOps (capability D)
Gate · Priority	G2 · Vertical slice
Stack	.NET
Environment	Bot permission to comment on work items
Depends on	BRF-01, BRF-04
v1.11	§3.5, §8.3

As an engineer, *I want* the brief posted as a comment on my ticket, *so that* I read it where I already work.

Acceptance criteria

- One comment per run, with the run ID and status: published, partial or blocked.
- Evidence links open the source item in ADO, Git or Confluence.
- VisionIQ's own comments never trigger a new run.

Tasks

Task	Technology	ID
ADO ticket brief publisher	.NET, ADO REST	S04-03
Publish result to ADO	.NET	S10-08
Post structured ticket brief	.NET, ADO comment formatting	S13-04

Completion evidence: screenshot of the brief on both live tickets.

BRF-06 · Rate each piece of advice

Field	Detail
Feature	Delivery in Azure DevOps (capability D)
Gate · Priority	G2 · Demo-critical
Stack	.NET
Environment	None beyond BRF-05
Depends on	BRF-05
v1.11	§7.4.1, §8.10

As an engineer, *I want* to mark each advice item as accepted, modified, rejected or not applicable, *so that* VisionIQ learns which advice helps.

Acceptance criteria

- Rating happens inside ADO, with no separate page.
- Each rating is stored against the advice ID with who and when.

Tasks

Task	Technology	ID
Record atomic advice outcomes	.NET, PostgreSQL	S10-09
Advice feedback mechanism	.NET, ADO	S13-05

Completion evidence: one rating given in ADO and the stored record with its advice ID.

Feature: Speed and hotspots

BRF-07 · Brief arrives within minutes

Field	Detail
Feature	Speed and hotspots (capability G)
Gate · Priority	G2, measured at G5 · Demo-critical
Stack	Python, Testing
Environment	None beyond G0
Depends on	BRF-05
v1.11	§11.1.1

As an engineer, *I want* the brief soon after I pick up the ticket, *so that* it helps before I start coding.

Acceptance criteria

- Time from pickup to posted brief is recorded for every run.
- It is measured separately from PR and CI time.

Tasks

Task	Technology	ID
Measure time-to-brief	Python, Cloud Logging	S10-11
Benchmark brief latency	pytest	S15-07

Completion evidence: median and p95 pickup-to-brief times over the test runs.

BRF-08 · Know the risky areas before tickets arrive

Field	Detail
Feature	Speed and hotspots (capability G)
Gate · Priority	G2 · Demo-critical
Stack	Python
Environment	None beyond G0
Depends on	LRN-09
v1.11	§3.4

As a team lead, *I want* a regular sweep of the areas with the most incidents, *so that* briefs and planning can flag hotspots early.

Acceptance criteria

- The sweep runs on a schedule and can be re-run on demand.
- Results are adjusted for activity, so busy areas aren't flagged just for being busy.

Tasks

Task	Technology	ID
Schedule periodic hotspot sweeps	Python, Cloud Scheduler	S10-12

Completion evidence: a hotspot report with the evidence behind each area.

> **Proposed BRF-08 subtasks (not yet in the master backlog)**
>
> PROP-07 Calculate activity-adjusted hotspot scores | Python, SQL
>
> PROP-08 Validate ranking, source evidence and false positives | pytest

Phase 3a — Defect

For a defect, VisionIQ proves the bug exists, proposes a bounded fix, proves the fix works without reopening old bugs, and opens a draft PR. If it can't verify something safely, it stops and says why. 6 stories in 3 features, gates G3–G4 (v1.11 §8.6, §8.7, capability group E).

Feature: Reproduce and fix

DEF-01 · Reproduce the defect before fixing it

Field	Detail
Feature	Reproduce and fix (capability E)
Gate · Priority	G3 · Vertical slice
Stack	Runner / CI, Python, Contracts
Environment	Decision D03 (demo repo and approved test runner); runner image built from the repo
Depends on	BRF-03, FND-04
v1.11	§8.6.2

As an engineer, *I want* VisionIQ to show a test failing on the current code for the reported reason, *so that* I know the fix targets the real bug.

Acceptance criteria

- The baseline commit and runner image are pinned before any test runs.
- The runner has no secrets and restricted network access.
- The test fails with the target failure signature; anything else is reported as UNAVAILABLE, FLAKY or ENVIRONMENT_ERROR and escalated.

Tasks

Task	Technology	ID
Fix request and response contracts	JSON Schema	S11-01
Safe workspace preparation	Cloud Build private pool or Cloud Run Job	S11-02
Defect reproduction check	Python	S11-03
Generate baseline failing test	Python, repo's test framework	S11-04

Completion evidence: the execution manifest and a red test log on the live defect.

DEF-02 · Propose a bounded fix

Field	Detail
Feature	Reproduce and fix (capability E)
Gate · Priority	G3 · Vertical slice
Stack	Python · AI models, .NET
Environment	Decision D04 (code may go to Claude)
Depends on	DEF-01, BRF-02
v1.11	§8.6.4

As an engineer, *I want* a small fix in the right layer, *so that* I can review it quickly and trust its scope.

Acceptance criteria

- An approved model proposes the patch inside an isolated workspace.

- Changes to security, payment, identity, schema or hardware areas, to secrets, or to dependency versions stop for engineer review.
- Iterations are capped; when the budget runs out the result is FAIL or INCONCLUSIVE, never green.

Tasks

Task	Technology	ID
Bounded patch proposal	Python, routed model	S11-05
Layer and restriction checks	Python	S11-06
Fix iteration budget	.NET	S11-13

Completion evidence: the proposed diff with its policy check result.

Feature: Verify

DEF-03 · Prove the fix works

Field	Detail
Feature	Verify (capability E)
Gate · Priority	G3 · Vertical slice
Stack	.NET, Runner / CI
Environment	A runnable workload for the touched path (D10)
Depends on	DEF-02
v1.11	§8.6.2, §8.7

As an engineer, *I want* the same failing test to pass on the fix, with performance checked, *so that* I know the fix works and doesn't slow anything down.

Acceptance criteria

- The same test passes on the candidate in the same pinned environment.
- The touched path is benchmarked with repeated paired runs; a noisy result is INCONCLUSIVE.
- Flaky results are rerun up to twice; the first failure is never erased.

Tasks

Task	Technology	ID
Execute candidate test	.NET orchestration, runner	S11-07
Performance baseline vs candidate	Runner, repo benchmark	S11-11
Flaky and environment failures	Runner	S11-12

Completion evidence: red-then-green logs for the same test, and the performance comparison.

DEF-04 · Old bugs don't come back

Field	Detail
Feature	Verify (capability E)
Gate · Priority	G3 · Demo-critical
Stack	.NET
Environment	At least one executable historical regression test in the demo area
Depends on	DEF-03, LRN-04
v1.11	§1.2, §4.5

As an engineer, *I want* past regression tests from this area re-run on the fix, *so that* I don't reopen a bug someone already fixed.

Acceptance criteria

- Historical regression tests are found through the ticket → PR → test links.
- An unsafe fix that passes the new test but fails an old one is caught before the PR.
- If no runnable old test exists, the brief and PR say so.

Tasks

Task	Technology	ID
Identify historical regression tests	.NET, PostgreSQL	S11-08
Execute historical regression replay	.NET, runner	S11-09

Completion evidence: an unsafe candidate failing a recovered old test, or a stated absence.

Feature: Draft PR and CI

DEF-05 · Draft PR with the full evidence trail

Field	Detail
Feature	Draft PR and CI (capabilities D, E)
Gate · Priority	G3 · Vertical slice
Stack	.NET
Environment	Bot identity with branch and PR rights only (HG-01)
Depends on	DEF-03, DEF-04, HG-01
v1.11	§8.6.1, §8.6.4

As a reviewer, *I want* the PR to explain defect → cause → risk → fix with links to the test runs, *so that* I can approve or reject it on evidence.

Acceptance criteria

- The draft PR is linked to the work item and created by the bot, which has no merge permission.
- The description cites commit SHAs, test IDs and run logs.
- PR checks link to the validation results.

Tasks

Task	Technology	ID
ADO PR creation adapter	.NET, ADO REST	S04-05
Create draft PR with evidence	.NET	S11-14
Expose PR validation links	.NET, ADO PR status API	S13-06

Completion evidence: the draft PR link for the live defect.

DEF-06 · Native CI runs on the PR, honestly

Field	Detail
Feature	Draft PR and CI (capabilities D, E)
Gate · Priority	G3 · Vertical slice
Stack	.NET, Testing
Environment	PR-triggered pipeline and branch policy on the demo repo
Depends on	DEF-05
v1.11	§8.6.1, §11.1.1

As a reviewer, *I want* the normal pipeline and branch policy to run on the PR, with honest results, *so that* green means green.

Acceptance criteria

- CI runs on the PR after it exists; results are published as PR checks.
- A required check that is failing, NOT_RUN or INCONCLUSIVE is never shown as green.
- A new commit on the PR invalidates earlier results.

Tasks

Task	Technology	ID
Execute touched-project suite	Azure Pipelines	S11-10
Invalidate stale checks	.NET, ADO PR status API	S11-17
Test quality gate truthfulness	xUnit, integration test	S15-12

Completion evidence: green required checks on the live defect PR, and a forced NOT_RUN check shown as not green.

Phase 3b — Enhancement

For an enhancement, VisionIQ gives guidance on what to watch out for and how to build it; a starter PR is optional, never required. 2 stories in 1 feature, gates G2–G3 (v1.11 §8.5, capability group F). The backlog has only one task here (S10-06); tasks marked **Proposed** need approval before they are added to it.

Feature: Guided build

ENH-01 · Guidance for building the enhancement

Field	Detail
Feature	Guided build (capability F)
Gate · Priority	G2 · Vertical slice
Stack	Python · AI models, .NET
Environment	None beyond G0
Depends on	BRF-03, BRF-05, LRN-07
v1.11	§8.5

As an engineer picking up an enhancement, *I want* guidance on contracts touched, past changes in this area, risks and tests to add, *so that* I build it right the first time.

Acceptance criteria

- The guidance covers the eight v1.11 sections: acceptance-criteria gaps, contracts and APIs touched, deployment and migration risk, related past changes, do / don't, recommended sequence, tests to add, reviewer decisions.
- A section with no evidence is marked missing, never filled with guesses.
- Each item cites its source and has an advice ID, like the defect brief.

Tasks

Task	Technology	ID
Generate enhancement brief	Python, Gemini	S10-06
Guidance schema with the eight §8.5 sections	Python, JSON Schema	PROP-01 (Proposed)
Contract and API impact from the dependency graph	Python, SQL	PROP-02 (Proposed)
Publish guidance on the ticket	.NET, ADO REST	PROP-03 (Proposed)

Completion evidence: guidance on the live enhancement ticket with all eight sections filled or marked missing.

ENH-02 · Optional starter PR

Field	Detail
Feature	Guided build (capability F)
Gate · Priority	G3 · Optional (after core)
Stack	Python · AI models, .NET
Environment	Bot identity with branch and PR rights only (HG-01)
Depends on	ENH-01, HG-01
v1.11	§2.1, §8.5

As an engineer, *I want* a starter PR when the change is well understood, *so that* I can begin from a scaffold instead of a blank branch.

Acceptance criteria

- A starter PR is created only when the engineer asks for it or policy allows it.
- It is a draft, created by the bot with no merge permission, and linked to the work item.
- Not creating a starter PR is never a failure: guidance alone completes the enhancement path.

Tasks

Task	Technology	ID
Decide when a starter PR is offered	Planning	PROP-04 (Proposed)
Generate starter scaffold	Python, routed model	PROP-05 (Proposed)
Create draft starter PR	.NET, ADO REST	PROP-06 (Proposed)

Completion evidence: a requested starter PR on the live enhancement, or the documented decision not to offer one.

Note: ENH-02 reuses the shared PR adapter (S04-05); it does not require DEF-05 to finish.

Phase 4 — Human Gate

The engineer decides; VisionIQ never merges. Every decision is recorded, and advice is tracked as right or wrong separately from whether it was accepted. 2 stories in 1 feature, gates G0–G3 (v1.11 §3.6, §8.3, §8.10, capability group D). What VisionIQ learns from those decisions lives in Learn → Continuous learning.

Feature: Engineer decision

HG-01 · The engineer decides; the bot never merges

Field	Detail
Feature	Engineer decision (capability D)
Gate · Priority	G0 setup, G3 enforced · Vertical slice
Stack	Azure DevOps setup, .NET
Environment	ADO project admin to set bot permissions and branch policy
Depends on	FND-01
v1.11	§8.3, §8.6.1

As a reviewer, *I want* VisionIQ's bot to be unable to approve or merge, *so that* every change that reaches main was approved by a person.

Acceptance criteria

- The bot can read, create branches and create PRs, nothing more.
- A test merge attempt by the bot is rejected by ADO.
- Change requests and rejections are recorded, and the PR stays open or closes as the reviewer chose.

Tasks

Task	Technology	ID
Define non-negotiable human review	ADO permissions, branch policy	S00-06
Enforce human gate	.NET	S11-15
Change-request and rejection handling	.NET, ADO REST	S11-16

Completion evidence: the bot's permission list, and the rejected merge attempt in the ADO audit log.

HG-02 · Track whether advice was right

Field	Detail
Feature	Engineer decision (capability D)
Gate · Priority	G2 · Demo-critical
Stack	.NET
Environment	None beyond G0
Depends on	BRF-06
v1.11	§7.4.1, §8.10

As a team lead, *I want* to see which advice turned out right or wrong, separately from whether engineers accepted it, *so that* we measure real accuracy, not popularity.

Acceptance criteria

- Engineer disposition (accepted, modified, rejected, not applicable) and evidence judgment (supported, contradicted, verified right, verified wrong, disputed) are stored separately.
- PR approval alone never marks advice as verified right.

Tasks

Task	Technology	ID
Record advice disposition events	.NET, PostgreSQL	S12-09
Record independent advice correctness	.NET, PostgreSQL	S12-10

Completion evidence: one advice item showing both states, with different values.

Impact Agent

The win statement asks for a history view inside the flagship Impact Agent. It depends on another team's interface, so the contract work starts in G0 even though the view is built in G5. 2 stories (v1.11 §8.4, capability group D).

IA-01 · Confirm the Impact Agent integration contract

Field	Detail
Feature	Impact Agent integration (capability D)
Gate · Priority	G0 · Demo-critical
Stack	.NET
Environment	Access to integration interface; decision D02
Depends on	None
v1.11	§8.4, §19

Technical story: confirm how Impact Agent can be extended and document its integration contract, before anything is built.

Acceptance criteria

- The extension mechanism, identity, payload and error handling are written down and agreed.
- If no access is possible, the contract is recorded as OPEN with the reason.

Tasks

Task	Technology	ID
Identify current Impact Agent integration interface	Document	S00-07
Inspect Impact Agent contract	.NET, API spec	S13-01

Completion evidence: the agreed contract document, or the OPEN record with its reason.

IA-02 · History view in the Impact Agent

Field	Detail
Feature	Impact Agent integration (capability D)
Gate · Priority	G5 · Demo-critical
Stack	.NET
Environment	Access to the Impact Agent as agreed in IA-01
Depends on	IA-01, LRN-09, BRF-05
v1.11	§8.4

As an engineer using the Impact Agent, *I want* to see the history VisionIQ found for my ticket there, *so that* I don't switch tools.

Acceptance criteria

- The view shows the same evidence as the brief, limited to what the engineer may see.
- The Impact Agent keeps working when VisionIQ is down.
- If access never arrives, it is reported as an unmet dependency; no substitute dashboard is built.

Tasks

Task	Technology	ID
Authorized history API	ASP.NET Core	S13-02
Integrate Impact Agent history view	.NET, Impact Agent interface	S13-03

Task	Technology	ID
Verify cross-project ACLs in the view	xUnit	S13-07
Handle VisionIQ outage in the view	.NET	S13-08

Completion evidence: the history view for the live defect inside the Impact Agent, or the stated unmet dependency.

Quality and Demo

This work proves VisionIQ works and shows it live. 4 stories, mostly gate G5; labelled test cases start in G1 (v1.11 §11, §12.1).

QA-01 · Measure accuracy on real past tickets

Field	Detail
Feature	Evaluation
Gate · Priority	G1 labels, G5 results · Demo-critical
Stack	Testing
Environment	15–30 resolved tickets with known correct history, labelled by the team
Depends on	LRN-05 for labels; LRN-09 and BRF-04 for results
v1.11	§11.2

As a judge, *I want* VisionIQ's accuracy measured on past tickets it hasn't seen, *so that* I can trust the results are not cherry-picked.

Acceptance criteria

- Labelled cases are prepared before any results are seen; a held-out set is kept apart.
- Replays use only history that existed at the time of each ticket.
- Results state their sample size, for example "right area in 18 of 24 tickets".

Tasks

Task	Technology	ID
Prepare labelled evaluation cases	Spreadsheet, fixtures	S15-01
Point-in-time backtesting	Python, pytest	S15-02
Benchmark candidate retrieval	pytest	S15-03
Evaluate citation correctness	pytest	S15-04
Evaluate root-cause and advice accuracy	pytest	S15-05
Evaluate Gemini vs Claude routing (after demo)	pytest	S15-06
Test retrieval scale and freshness (after demo)	pytest	S15-08

Completion evidence: results table with sample sizes and the snapshot ID used.

QA-02 · Safety and reliability tests

Field	Detail
Feature	Safety
Gate · Priority	G3, G5 · Demo-critical
Stack	Testing, Runner / CI
Environment	None beyond G0
Depends on	DEF-06, FND-05
v1.11	§9, §16

Technical story: prove the safety rules hold under failure: lost or duplicated events, runner misuse, outages and access boundaries.

Acceptance criteria

- Duplicate, late and missing events leave the system consistent.
- The runner cannot read secrets or reach the network.

- A security and access sign-off is recorded before the demo.

Tasks

Task	Technology	ID
Event reliability chaos test	pytest, Pub/Sub	S15-09
PR-runner safety test	Runner	S15-11
Verify availability and failure fallback	Cloud Run	S17-11
Security and ACL sign-off	Document	S17-12

Completion evidence: test report for each case, and the signed-off checklist.

QA-03 · Release to Google Cloud

Field	Detail
Feature	Release
Gate · Priority	G5 · Demo-critical
Stack	GCP / DevOps
Environment	None beyond G0
Depends on	FND-04, LRN-05
v1.11	§10.8

Technical story: deploy a tagged release with the knowledge base loaded, so the demo runs on the real environment.

Acceptance criteria

- A tagged release candidate is deployed from the pipeline, not by hand.
- The knowledge base is backfilled in the demo environment before rehearsal.

Tasks

Task	Technology	ID
Tagged release candidate	Git tags, Azure Pipelines	S17-01
Deploy Cloud SQL and backfill knowledge	Cloud SQL, Cloud Run Jobs	S17-02
Deploy .NET and Python runtime	Cloud Run	S17-03

Completion evidence: release tag, deploy log and backfill counts in the demo environment.

QA-04 · Live demonstration, end to end

Field	Detail
Feature	Demo
Gate · Priority	G5 · Vertical slice for the three live runs; Demo-critical for the rest
Stack	Demo, Testing
Environment	Live tickets and demo environment ready (FND-01, QA-03)
Depends on	BRF-05, DEF-06, HG-01, ENH-01; IA-02 and LRN-13 for their parts
v1.11	§2.1, §12.1

As a judge, *I want* to watch a live defect and a live enhancement go through VisionIQ, *so that* I see the win condition met, not described.

Acceptance criteria

- The defect ends as an engineer-approved PR with green tests; the enhancement gets guidance.

- The Impact Agent view and a replayed learning cycle are shown, or the unmet dependency is stated.
- The demo is recorded, rehearsed twice and frozen before submission.

Tasks

Task	Technology	ID
Execute complete win scenario	Integration test	S15-13
Live event pipeline smoke test	ADO, Cloud Run	S17-05
Live defect fix demonstration	ADO, runner	S17-06
Live enhancement demonstration	ADO	S17-07
Impact Agent history demonstration	Impact Agent	S17-08
Replay continuous learning demonstration	Fixtures	S17-09
Record and edit demo	Screen recording	S17-13
Submission assets and links	Document	S17-14
Final rehearsal and freeze	Git tag	S17-15

Completion evidence: the recorded demo and the submission package.

Reporting

Reporting is a cross-phase output, not a phase. What a report must contain is settled; how it is delivered is still open as decision D11. Judge access, including whether Google accounts are available, must be confirmed. 2 stories, gate G5 (pending addition to v1.11 §19).

RPT-01 · Report what VisionIQ did, from stored facts

Field	Detail
Feature	Reporting
Gate · Priority	G5 · Demo-critical
Stack	Database, .NET, Python
Environment	None beyond G0
Depends on	BRF-06, DEF-06, QA-01
v1.11	Not yet (D11)

As a judge, *I want* a report of what VisionIQ found, changed and verified for each ticket, plus overall accuracy, *so that* I can see it solved the problem.

Acceptance criteria

- Every number and pass/fail result comes from stored run data, never from an AI summary.
- Failed, skipped and unknown checks are shown as such.
- Each figure can be traced back to its run ID or test result.

Tasks

Task	Technology	ID
Per-ticket report schema	PostgreSQL	S14-01
Aggregate impact metrics	Python, SQL	S14-02
Report data projection	.NET, SQL	S14-03
Show failed and unknown checks explicitly	.NET	S14-08
Record report provenance	.NET, PostgreSQL	S14-10
Generate final accuracy and impact report	Stored evaluation results	S17-10

Completion evidence: the report data for both live tickets, each figure traceable to a run ID.

RPT-02 · Deliver the report (pending D11)

Field	Detail
Feature	Reporting
Gate · Priority	G5 · Pending D11
Stack	Set by D11
Environment	Decision D11
Depends on	RPT-01
v1.11	Not yet (D11)

Technical story: deliver reports to engineers and judges in the way D11 decides.

Acceptance criteria

- Defined once D11 is decided.
- Whatever the choice, a report link never replaces the Impact Agent history view.

Tasks

Task	Technology	ID
Read-only report viewer	Vue (only if D11 chooses it)	S14-04
Static HTML or PDF export	.NET	S14-05
Report authorization	.NET	S14-06
Publish report links	.NET, ADO REST	S14-07
Judge-facing scorecard example	Vue (only if D11 chooses it)	S14-09
Deploy report viewer and exports	Cloud Run	S17-04

Completion evidence: defined with D11.

Extensions (optional)

These stories hold the 17 backlog tasks marked Extension. None is needed for the win condition; build one only after the vertical slice and demo-critical stories work, and only if it adds measurable value. 5 stories, gate After.

EXT-01 · Jira as a ticket source

Field	Detail
Feature	Extensions (capability A)
Gate · Priority	After · Extension
Stack	.NET
Environment	A Jira site with authorized access
Depends on	LRN-01, BRF-05
v1.11	§4.9

As an engineer on a team that uses Jira, *I want* the same brief on my Jira issue, *so that* VisionIQ works beyond Azure DevOps.

Acceptance criteria

- Jira issues map to the same canonical records as ADO work items.
- Briefs post to Jira the same way they post to ADO.
- Access rules from Jira are respected.

Tasks

Task	Technology	ID
Jira work-item connector	.NET, Jira REST	S04-06
Jira event and brief adapter	.NET, Jira REST	S04-07
Extend provider integrations (more ADO orgs, Jira sites, Confluence tenants)	.NET, connector APIs	S18-02

Completion evidence: a brief posted on a Jira issue from Jira history.

EXT-02 · Export lessons as OKF

Field	Detail
Feature	Extensions (capability C)
Gate · Priority	After · Extension
Stack	Python
Environment	ENV-18 decision that OKF adds value
Depends on	LRN-10, LRN-13
v1.11	§6.20

Technical story: export approved lessons as versioned OKF Markdown and YAML, as a curated copy, not as the database or the search engine.

Acceptance criteria

- Only approved lessons are exported, each with its sources and version.
- Stale or superseded lessons are flagged in the export.
- Turning the export off changes nothing in the core flow.

Tasks

Task	Technology	ID
Design OKF bundle structure	OKF, Markdown, YAML	S09-05
Generate OKF from curated lessons	Python, Jinja2	S09-06
Validate OKF provenance and freshness	Python	S09-07
Store and version OKF bundles	Cloud Storage, Git	S09-08
Make OKF searchable	pgvector, full-text	S09-09
Expand OKF curation controls	OKF, Git, ADO	S18-04

Completion evidence: one exported bundle that validates and links back to its lessons.

EXT-03 · Evaluate Google ADK

Field	Detail
Feature	Extensions (capability C)
Gate · Priority	After · Extension
Stack	Python · AI models
Environment	ENV-18 decision to evaluate ADK
Depends on	BRF-02, BRF-03
v1.11	§10.8, §10.9

Technical story: try ADK for bounded retrieval or investigation steps, and keep it only if it is simpler or measurably better than the existing orchestration.

Acceptance criteria

- A comparison records quality, latency, cost and recoverability against the current flow.
- ADK is never used for ingestion, access checks, CI verdicts, merges or publishing lessons.
- The core flow runs with ADK switched off.

Tasks

Task	Technology	ID
Prototype Google ADK workflow	Google ADK	S08-09
Expand ADK orchestration where beneficial	Google ADK	S18-05

Completion evidence: the comparison and the keep-or-drop decision.

EXT-04 · Wider code coverage and scale

Field	Detail
Feature	Extensions (capabilities B, G)
Gate · Priority	After · Extension
Stack	Python, .NET, Testing
Environment	Repositories in languages the demo did not cover
Depends on	LRN-06, LRN-09
v1.11	§4.7, §6.17

Technical story: support more repository languages, larger indexes and long-running checks without weakening access rules.

Acceptance criteria

- Each new language adapter keeps the file-level fallback.
- Index or storage changes are justified by a benchmark.

- Production reconciliation repairs gaps between source systems and VisionIQ.

Tasks

Task	Technology	ID
Additional repo language adapters	tree-sitter, native parsers	S18-01
Scale graph and search indexing	PostgreSQL, load tests	S18-03
Cross-project performance baseline	CI, analytics	S18-07
Production reconciliation jobs	Cloud Scheduler, Cloud Run Jobs	S18-08

Completion evidence: benchmark results for each change.

EXT-05 · Engineering workspace and Time Machine

Field	Detail
Feature	Extensions (capability C)
Gate · Priority	After · Extension
Stack	Vue
Environment	Product decision that a separate workspace is wanted
Depends on	LRN-07, LRN-09
v1.11	§4.2, §4.9

As a team lead, *I want* to explore the knowledge base and see a component's history as of any date, *so that* I can investigate patterns outside a single ticket.

Acceptance criteria

- It is optional; the ticket workflow never depends on it.
- It shows only what the viewer may see, as of the chosen date.

Tasks

Task	Technology	ID
Enterprise intelligence workspace	Vue, API	S18-06
Engineering Time Machine exploration	Temporal SQL, Vue	S18-09

Completion evidence: an as-of-date history view for one component.

Proposed tasks — approval needed

Six enhancement tasks and two hotspot tasks are proposed; none is in the current master backlog. The enhancement tasks implement §8.5; hotspot tasks complete BRF-08 scoring and verification. Approve the proposed tasks before adding them to the master backlog.

ID	Story	Proposed task	Technology	Why it is needed
PROP-01	ENH-01	Guidance schema with the eight §8.5 sections	Python, JSON Schema	v1.11 requires all eight sections
PROP-02	ENH-01	Contract and API impact from the dependency graph	Python, SQL	Contract impact is a required section with no task
PROP-03	ENH-01	Publish guidance on the ticket	.NET, ADO REST	Guidance must reach the ticket like the defect brief
PROP-04	ENH-02	Decide when a starter PR is offered	Planning	v1.11 makes the starter PR optional; the trigger is undecided
PROP-05	ENH-02	Generate starter scaffold	Python, routed model	No task builds the scaffold
PROP-06	ENH-02	Create draft starter PR	.NET, ADO REST	No task creates the starter PR

Additional BRF-08 proposals - hotspot scoring

ID	Story	Task and technology
PROP-07	BRF-08	Calculate activity-adjusted hotspot scores - Python, SQL
PROP-08	BRF-08	Verify rankings and source evidence - Python, pytest

Proposed tasks are not part of the original 236 until approved.

Build order

Build the vertical slice through every gate first, then add the demo-critical stories gate by gate. Each gate ends with something demonstrable, as its v1.11 exit condition requires.

1. **G0 — Development readiness:** FND-01 to FND-05, HG-01 (bot permissions), IA-01 (contact the Impact Agent integration in week one).
2. **G1 — Trustworthy history:** LRN-01, LRN-03, LRN-04, LRN-05, LRN-06, then LRN-02, LRN-07, LRN-10, LRN-11 and QA-01 labelled cases.
3. **G2 — Learn plus automatic brief:** LRN-08, LRN-09, BRF-01 to BRF-05, ENH-01, then BRF-06 to BRF-08, HG-02, LRN-12.
4. G3 — Defect PR and human gate: DEF-01, DEF-02, DEF-03, DEF-04, DEF-05, DEF-06; then LRN-13 and QA-02. ENH-02 is optional.
5. G4 — Failure-memory demonstration: replay a fix blocked by DEF-04 checks.
6. **G5 — Measure and demo:** QA-01 results, IA-02, RPT-01, QA-03, QA-04, FND-06 hardening, and RPT-02 once D11 is decided.
7. **After the hackathon:** LRN-14, the after-demo tasks inside other stories, and any extension story (EXT-01 to EXT-05) the team chooses.

If time runs short, protect G2 (briefs on both live tickets) and G3 (the defect PR) first: together they meet the core of the win condition.